

Video Recording Script: Emergency Ambulance Dispatch System

Target Time: ~7 Minutes 30 Seconds

Prerequisites to keep open before recording:

1. **Browser Tab:** <https://ambulance-dispatch-backend-66f2.onrender.com/health>
2. **VS Code:** Project root opened with file tree visible
3. **Postman:** Emergency Ambulance Dispatch System API collection imported

Timeline Cheat-Sheet

Timestamp	Topic	Screen / Tool
0:00 – 1:00	1. Introduction & Architecture	Browser & VS Code
1:00 – 2:30	2. Role-Based Access Demo (403 Forbidden)	Postman
2:30 – 4:15	3. CRUD Operations Demo (Ambulance Resource)	Postman
4:15 – 5:15	4. Validation & Error Handling Demo (422, 404, 401)	Postman
5:15 – 6:30	5. Payment Flow Demo (Stripe & Webhook)	Postman & Browser
6:30 – 7:30	6. Technical Challenge (Race Conditions & Double-Booking)	VS Code (<code>dispatch.service.ts</code>)
7:30 – 8:00	7. Automated Tests & Wrap-Up	Terminal (<code>npm test</code>)

1. Introduction & Architecture (45–60 seconds)



What I show:

Start on the browser showing the live `/health` endpoint on Render. Then switch to VS Code with the file explorer open showing `src/modules`.



What I say:

"Hello everyone. Welcome to the demo of the Emergency Ambulance Dispatch System API.

In medical emergencies, manual dispatch is slow and prone to human error. This backend solves that problem. It automates emergency requests, pairs patients with ambulances and drivers, tracks trip progress in real-time, and handles payments.

The API is live on Render, connected to a Neon PostgreSQL database and Redis Cloud.

Let's look at the architecture in VS Code.

I built this backend using a clean, layered modular pattern.

First, **Routes** define endpoints and apply authentication, RBAC, and Zod validation.

Second, **Controllers** parse incoming requests and send unified JSON responses.

Third, **Services** hold all the core business logic.

And fourth, **Prisma ORM** interacts directly with our PostgreSQL database with full type safety.

This separation keeps the codebase clean, modular, and easy to maintain."

2. Role-Based Access Demo (60–90 seconds)



What I show:

Switch to Postman. Open `01. Auth Module`. Show `Admin Login`, `Patient Login`, then send an Admin-only request using the Patient token.



What I say:

"Now let's look at Role-Based Access Control.

We have three distinct roles: `PATIENT` , `DRIVER` , and `ADMIN` .

First, I will log in as an `ADMIN` .

[Click Send on 'Admin Login']

We get our JWT access token. My test script automatically saves this token in Postman.

As an Admin, I have full permissions. Let's fetch the system statistics.

[Send 'GET /api/v1/admin/stats']

Status 200 OK. The Admin can see fleet metrics and revenue.

Now, let's switch users. I will log in as a `PATIENT` .

[Click Send on 'Patient Login']

The active token is now updated to the Patient.

Let's see what happens if this Patient tries to access an Admin endpoint, like toggling a user's active status.

[Send 'PATCH /api/v1/admin/users/{{targetUserId}}/status']

Look at the response: **HTTP 403 Forbidden**. Message: 'Forbidden: Insufficient permissions'.

Finally, let's log in as a `DRIVER` .

[Click Send on 'Driver Login']

As a driver, I can fetch my ongoing assigned emergency dispatch.

[Send 'GET /api/v1/dispatches/my-active']

Status 200 OK. But if this Driver tries to access Admin user management, they also get blocked with a 403 Forbidden.

Our RBAC middleware checks the role stored inside the verified JWT on every single protected route. If the role doesn't match, it blocks the request immediately."

3. CRUD Operations Demo (90–120 seconds)



What I show:

In Postman, execute these 4 requests in order:

1. `POST /api/v1/ambulances`
2. `GET /api/v1/ambulances`
3. `PATCH /api/v1/ambulances/:id`
4. `DELETE /api/v1/ambulances/:id`

What I say:

"Now let's demonstrate full CRUD operations using our Ambulance resource. I will switch back to my Admin token.

First, CREATE with POST.

[Send 'POST /api/v1/ambulances']

I send vehicle number 'AMB-TEST-99', type 'ADVANCED_LIFE_SUPPORT', and status 'AVAILABLE'.

Status 201 Created. The new ambulance is created with a generated UUID, and our test script saves its ID.

Second, READ with GET.

[Send 'GET /api/v1/ambulances?page=1&limit=10']

Status 200 OK. We get a paginated list of ambulances along with pagination metadata: page, limit, total, and totalPages.

Third, UPDATE with PATCH.

[Send 'PATCH /api/v1/ambulances/{{ambulancelid}}' with body {"status": "MAINTENANCE"}]

Status 200 OK. The status changes to 'MAINTENANCE'. We also log this update in our AuditLog table.

Fourth, DELETE.

[Send 'DELETE /api/v1/ambulances/{{ambulancelid}}']

Status 200 OK. We do not hard delete records. We use soft deletes. The API sets a `deletedAt` timestamp.

If we fetch this ambulance again, the backend automatically filters it out."

4. Validation & Error Handling Demo (45–60 seconds)

What I show:

In Postman:

1. Send `POST /api/v1/auth/register` with an invalid email.
2. Send `GET /api/v1/ambulances/invalid-uuid`.
3. Send a request with no Authorization header.

What I say:

"Next, let's look at validation and error handling.

I use **Zod** to validate request bodies, URL params, and query filters before they ever hit the controllers.

Let's test a validation error. I will send a registration request with an invalid email address.

[Send 'POST /api/v1/auth/register' with "email": "not-an-email"]

Status **422 Unprocessable Entity**. The response returns a clean error array showing the exact field 'body.email' and the message 'Invalid email address'.

Now let's trigger a **404 Not Found**. I will query an ambulance using a non-existent UUID.

[Send request with fake UUID]

Status **404 Not Found**. We get a clean, standardized error message.

Finally, let's trigger a **401 Unauthorized** by removing the Bearer token entirely.

[Disable Authorization header and send]

Status **401 Unauthorized**.

All errors pass through our centralized `errorHandler` middleware. This guarantees a consistent JSON format across the entire API."

5. Payment Flow Demo (60–90 seconds)

What I show:

In Postman:

1. Show `GET /api/v1/payments/fare/:requestId` .
2. Send `POST /api/v1/payments/stripe/checkout` .
3. Copy `checkoutUrl` , open it in the browser, show the Stripe test page.
4. Send the Stripe Webhook request in Postman.
5. Show `GET /api/v1/payments/:requestId` showing `SUCCESS` .

What I say:

"Now let's look at the payment flow.

Once an ambulance completes a trip, the patient pays.

First, the patient requests a fare calculation.

[Send 'GET /api/v1/payments/fare/:requestId']

The system calculates the base fare, ambulance type surcharge, and distance breakdown.

Next, the patient initiates a Stripe checkout session.

[Send 'POST /api/v1/payments/stripe/checkout']

Status 201 Created. The backend creates a Payment record in 'PENDING' status and returns a hosted Stripe checkoutUrl .

[Open the checkoutUrl in the browser]

This takes the patient to Stripe's secure payment page, where they can pay using a test card.

Once the payment succeeds, Stripe notifies our backend via a signed webhook.

[Send 'POST /api/v1/payments/stripe/webhook']

Our webhook verifies the cryptographic signature from Stripe. It marks the Payment as 'SUCCESS' and automatically marks the EmergencyRequest as 'COMPLETED'.

Let's verify by fetching the payment record.

[Send 'GET /api/v1/payments/:requestId']

Status 200 OK. The payment status is now 'SUCCESS'.

We also support bKash Tokenized Checkout and SSLCommerz using this same pattern."

6. Technical Challenge (45–60 seconds)



What I show:

Switch to VS Code. Open `src/modules/dispatch/dispatch.service.ts` around line 20. Highlight `prisma.$transaction` .



What I say:

"Now I want to share a key technical challenge I solved: **Preventing Double-Booking and Race Conditions in Dispatch**.

In an emergency system, multiple dispatchers can try to assign the same ambulance to different emergencies at the exact same moment. If two requests get the same ambulance, someone in

critical condition doesn't get help.

To solve this, I used **Prisma Interactive Database Transactions**.

Here in `dispatch.service.ts` , everything runs inside `prisma.$transaction` .

Inside the transaction:

1. We check if the ambulance is currently `AVAILABLE` .
2. We check if the driver is marked `isAvailable` .
3. If valid, we create the `Dispatch` record, create the initial `TripStatusLog` , update the `EmergencyRequest` to `DISPATCHED` , update the ambulance to `DISPATCHED` , and set `driver.isAvailable` to `false` .

All five database operations succeed together, or all of them roll back. This completely eliminates race conditions and guarantees zero double-booking."

7. Automated Tests & Wrap-Up (30 seconds)



What I show:

Open the VS Code integrated terminal. Type `npm test` and hit Enter. Let all tests run and turn green.



What I say:

"Finally, let's verify our automated test suite.

[Run `npm test`]

We have **65 automated tests** written in **Vitest**. They cover unit tests, validation schemas, live database integration tests, and full End-to-End emergency lifecycles.

As you can see, all 10 test suites and 65 tests pass cleanly.

Thank you for your time and for reviewing my project!"